

In Class Assignment Three

In the following assignment you will be asked to fill in python code and derivations for a number of different problems. Please read all instructions carefully and turn in the rendered notebook (or HTML of the rendered notebook) before the end of class (or right after class). The initial portion of this notebook is given before class and the remainder is given during class. Please answer the initial questions before class, to the best of your ability. Once class has started you may rework your answers as a team for the initial part of the assignment.

Contents

- Loading KDDCup Data
- The remaining links will be active for the in-class assignment:
- KDDCup Evaluation and Cross Validation
- More Cross Validation
- Statistical Comparison

Before coming to class, please make sure you have the latest version of `scikit-learn`. This notebook was created for version 0.18 and higher.

[Back to Top](#)

Loading KDDCup Data

Please run the following code to read in the “KDD Cup” dataset from sklearn’s data loading module. It consists of examples of different simulated attacks for the 1998 DARPA Intrusion Detection System (IDS).

This will load the data into the variable `ds`. `ds` is a `bunch` object with fields like `ds.data` and `ds.target`. The field `ds.data` is a numpy matrix of the continuous features in the dataset. **The object is not a pandas dataframe. It is a numpy matrix.** Each row is a set of observed instances, each column is a different feature. It also has a field called `ds.target` that is an integer value we are trying to predict (i.e., a specific integer represents a specific person). Each entry in `ds.target` is a label for each row of the `ds.data` matrix.

```
library(reticulate)
py_require("matplotlib")
py_require("numpy")
py_require("scikit-learn")

# fetch the dataset
from sklearn.datasets import fetch_kddcup99
from sklearn import __version__ as sklearn_version

print('Sklearn Version:', sklearn_version)

## Sklearn Version: 1.7.2

ds = fetch_kddcup99(subset='http')
print(ds.data)

## [[-2.3025850929940455  5.199049364889368  8.60338923611281]
##  [-2.3025850929940455  5.476881874464279  6.1864143640514095]
##  [-2.3025850929940455  5.460010955546024  7.198258368620619]
##  ...
##  [-2.3025850929940455  5.313698468586339  7.090160165637395]
##  [-2.3025850929940455  5.6736668507515775  7.090160165637395]
##  [-2.3025850929940455  5.3895282466014205  7.118097238459137]]
```

```

import numpy as np
# get some of the specifics of the dataset
X = ds.data
y = np.array(ds.target != b'normal.')
y = y.astype(int)

n_samples, n_features = X.shape
n_classes = len(np.unique(y))

print("n_samples: {}".format(n_samples))

## n_samples: 58725
print("n_features: {}".format(n_features))

## n_features: 3
print("n_classes: {}".format(n_classes))

## n_classes: 2

```

Question 1: How many instances are in the binary classification problem loaded above? How many instances are in each class? **Plot a pie chart or bar chart of the number of classes.**

```

from matplotlib import pyplot as plt
plt.style.use('ggplot')

=== Fill in code below=====

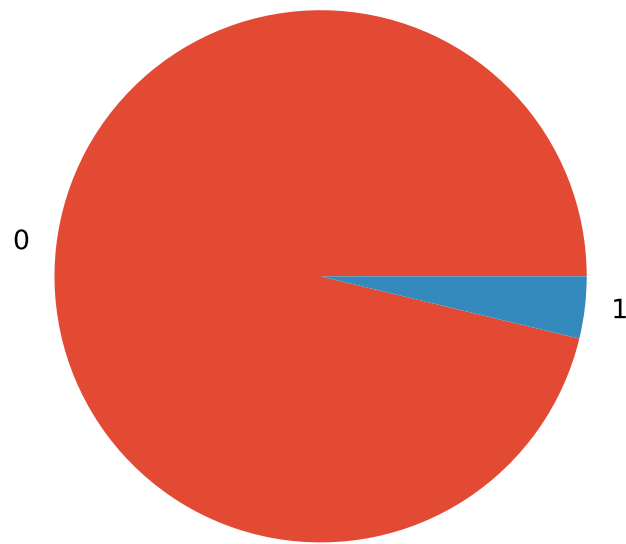
arange = np.arange(n_classes).astype('str')
counts = np.bincount(y)

print('Number of instances in each class:', counts)

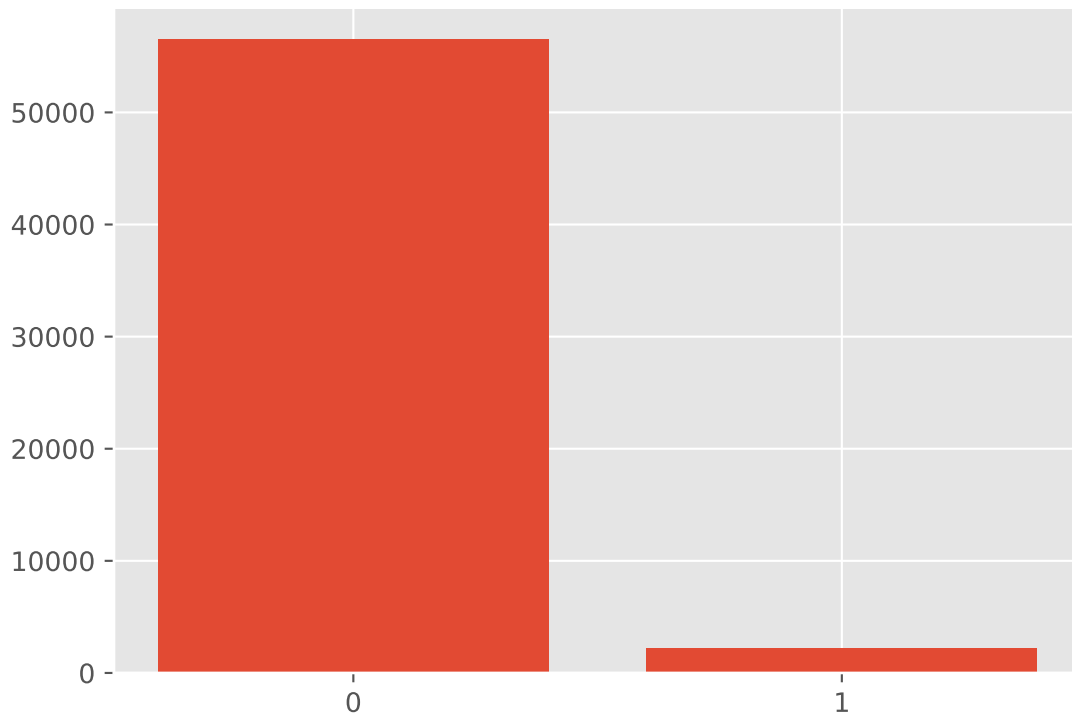
## Number of instances in each class: [56516  2209]
plt.pie(counts, labels = arange)

## ([<matplotlib.patches.Wedge object at 0x1467692b0>, <matplotlib.patches.Wedge object at 0x3006604d0>],
plt.show()

```



```
plt.bar(arange, counts)  
plt.show()
```



Video notes

Code sample imports

```
from pprint import pprint
from sklearn.model_selection import train_test_split, \
    learning_curve, validation_curve, cross_val_score, \
    GridSearchCV, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.decomposition import PCA
from sklearn.pipeline import Pipeline
from sklearn.metrics import classification_report, \
    precision_score, recall_score, f1_score, make_scorer, roc_curve

def plot_filled(train_scores, test_scores, train_x_axis, xlabel=''):
    train_mean = np.mean(train_scores, axis=1)
    train_std = np.std(train_scores, axis=1)
    test_mean = np.mean(test_scores, axis=1)
    test_std = np.std(test_scores, axis=1)

    plt.plot(train_x_axis, train_mean,
             color='blue', marker='o',
             markersize=5, label='training accuracy')

    plt.fill_between(train_x_axis,
```

```

        train_mean + train_std,
        train_mean - train_std,
        alpha=0.15, color='blue')

plt.plot(train_x_axis, test_mean,
         color='green', linestyle='--',
         marker='s', markersize=5,
         label='validation accuracy')

plt.fill_between(train_x_axis,
                 test_mean + test_std,
                 test_mean - test_std,
                 alpha=0.15, color='green')

plt.grid()
plt.xlabel(xlabel)
plt.ylabel('Accuracy')
plt.legend(loc='lower right')
plt.tight_layout()

```

Pipeline overview (0:00-10:33)

- How reliable are our estimates of performance?
- Performance of a model may depend on other factors besides the learning algorithm:
 - Class distribution
 - Cost of misclassification
 - Size of training and test sets

Best practices:

- Combine pre- and post-processing into stages with pipelines
- Excellent way to prevent data snooping
 - Guarantees separation of testing and training sets
 - Example stages:
 - * Scaling
 - * Dimensionality Reduction
 - * Learning Algorithm
 - * Predictive Model

```

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.20, random_state=1)

pipe_lr = Pipeline([('scl', StandardScaler()),
                    ('pca', PCA(n_components = 2)),
                    ('clf', LogisticRegression(random_state = 1))])

pipe_lr.fit(X_train, y_train)

## Pipeline(steps=[('scl', StandardScaler()), ('pca', PCA(n_components=2)),
##                ('clf', LogisticRegression(random_state=1))])
print("Test Accuracy: %0.3f" % pipe_lr.score(X_test, y_test))

## Test Accuracy: 1.000
y_pred = pipe_lr.predict(X_test)

```

Use a testing set, and *never, never, never* let the model see it. Strategies:

- Holdout
 - Reserve $x\%$ for training and $(1 - x)\%$ for testing
 - For instance, 80% for training and 20% for testing
 - Further split training into training and validation - useful for tuning hyperparameters without data snooping
 - Train, tune, and evaluate with training and validation sets
 - Final performance comes only from test set
- Random subsampling
 - Repeated holdout, with replacement
- Cross Validation (preferable to others in medium-size and smaller datasets)
 - Partition data into k disjoint subsets
 - k -fold: train on $k - 1$ partitions, test on the remaining one (often chosen to be 10 or sometimes 5, depending on size of dataset)
 - * For instance, divide data into 10 folds, and for 10 iterations, choose a different fold to be the test fold
 - * Then we can average the errors as well as get a variance of it
 - Leave-one-out: $k = M$ (usually too computationally intensive)
- Stratified Cross Validation
 - Select samples, keeping overall class distribution same for each fold
 - Same as k -fold but stratify folds by classes
 - We don't want folds that are entirely the majority class!

Time series analysis (10:34-18:06)

TL;DR: The primary point of performing cross validation on a dataset is **to understand how the algorithm might perform on data it was not trained with, as well as the variance in performance**. The secondary point is hyperparameter tuning: use the average performance from CV for the final model.

Therefore the situation must be a plausible deployment scenario, so that we have an idea of the hyperparameters and performance of using the entire dataset.

- Folds must be plausible, representative of the actual use case of a classifier.
- For instance, for time series data, you cannot apply stratified cross validation.
 - Points near each other in time might be very similar, solely due to how close they are in time, and not for predictive value.
 - In practice, you can't interpolate between current and future points to obtain future data.
- Similarly, suppose we fold the data over time.
 - Although it's not as bad as random stratification, we still end up with cross-validation cases where $T1 < T2 < T3$, and we use $T1$ and $T3$ to predict $T2$!
- Ideally, we take a fold of data, and predict the next point.
 - Then, add the next point and train on that, and so on!

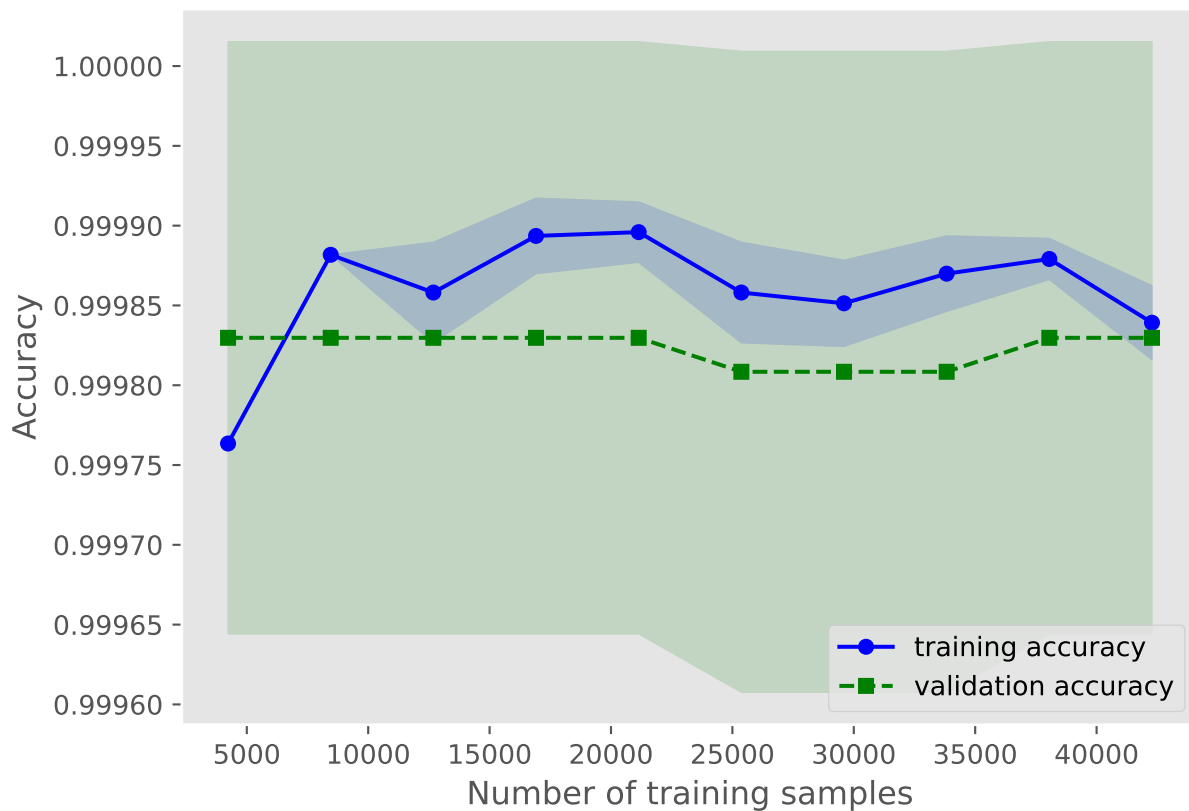
Intro to Learning Curve (18:07-25:00)

- How do we know a model is actually good?
 - We need to have low bias (accurate) *and* low variance (precise)
 - Complex models can really fit the training data, giving lower bias
 - Simpler models have trouble fitting data, resulting in higher bias
 - But complex models can have high variance in their decision!
- How do we estimate bias and variance for a classifier?
 - The easiest way to do this is with a learning curve. If we train a lot of times, we can plot accuracy vs. sample size, as well as include a measure of spread for the accuracy.
 - You can't estimate the curve without collecting the data. Some bounds are known but they are too loose to be useful.

Learning Curve (25:00-27:08)

```
pipe_lr = Pipeline([('scl', StandardScaler()),
                    ('pca', PCA(n_components = 2)),
                    ('clf', LogisticRegression(random_state = 0))])
train_sizes, train_scores, test_scores = learning_curve(
    estimator = pipe_lr, X = X_train, y = y_train,
    train_sizes = np.linspace(0.1, 1.0, 10),
    cv = 10, n_jobs = -1)

plot_fitted(train_scores, test_scores, train_sizes,
            xlabel='Number of training samples')
plt.show()
```



Validation Curve (27:08-34:10)

```
param_names = pipe_lr.get_params()
pprint(param_names)

## {'clf': LogisticRegression(random_state=0),
##  'clf__C': 1.0,
##  'clf__class_weight': None,
##  'clf__dual': False,
##  'clf__fit_intercept': True,
##  'clf__intercept_scaling': 1,
##  'clf__l1_ratio': None,
```

```

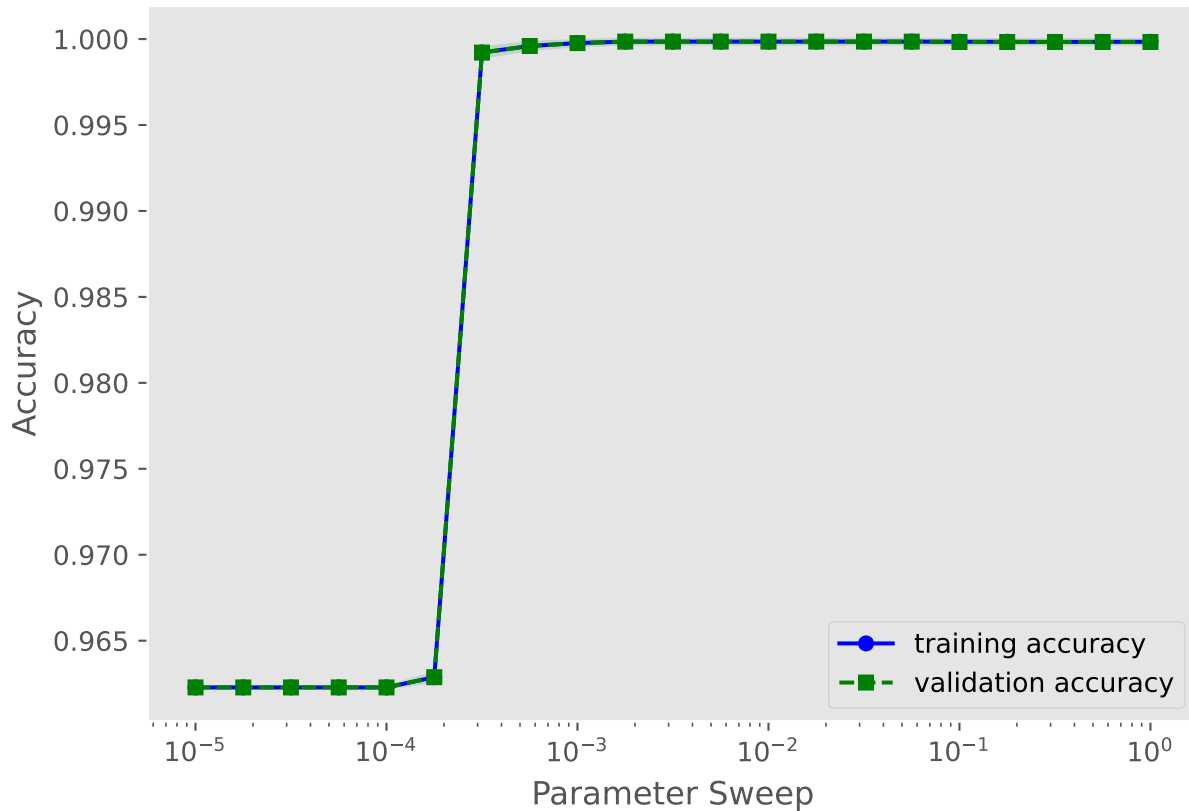
## 'clf__max_iter': 100,
## 'clf__multi_class': 'deprecated',
## 'clf__n_jobs': None,
## 'clf__penalty': 'l2',
## 'clf__random_state': 0,
## 'clf__solver': 'lbfgs',
## 'clf__tol': 0.0001,
## 'clf__verbose': 0,
## 'clf__warm_start': False,
## 'memory': None,
## 'pca': PCA(n_components=2),
## 'pca__copy': True,
## 'pca__iterated_power': 'auto',
## 'pca__n_components': 2,
## 'pca__n_oversamples': 10,
## 'pca__power_iteration_normalizer': 'auto',
## 'pca__random_state': None,
## 'pca__svd_solver': 'auto',
## 'pca__tol': 0.0,
## 'pca__whiten': False,
## 'scl': StandardScaler(),
## 'scl__copy': True,
## 'scl__with_mean': True,
## 'scl__with_std': True,
## 'steps': [('scl', StandardScaler()),
##           ('pca', PCA(n_components=2)),
##           ('clf', LogisticRegression(random_state=0))],
## 'transform_input': None,
## 'verbose': False}

```

```

param_range = np.logspace(-5, 0, 5 * 4 + 1)
train_scores, test_scores = validation_curve(
    estimator=pipe_lr,
    X=X_train,
    y=y_train,
    param_name='clf__C',
    param_range=param_range,
    cv=10, n_jobs=-1)
plot_filled(train_scores, test_scores, param_range, xlabel='Parameter Sweep')
plt.xscale('log')
plt.show()

```



Note: This is no longer a validation set as we have used it to tune the hyperparameters of the algorithm, thus we have potentially introduced data snooping.

This is why we want to divide into a training and validation fold, and we can select the best out of several tested parameters in a grid search. It's called a grid search because we look at every pair of values.

```
param_grid = {
    'pca__n_components': [1, 3],
    'clf__C': [0.00001, 0.0001, 0.001, 0.01, 0.1, 1.0,]
}
gs = GridSearchCV(estimator=pipe_lr,
    param_grid=param_grid,
    scoring='accuracy',
    cv = 2)

scores = cross_val_score(gs, X_train, y_train, scoring='accuracy', cv = 5)
print('CV accuracy: %0.3f \u00b1 %0.3f' % (np.mean(scores), np.std(scores)))
```

```
## CV accuracy: 1.000 ± 0.000
```

Evaluation Metrics (34:10 - 41:44)

- Focus on the predictive capability of the model, not on how fast it takes to classify/build, scalability, etc.
- For instance, a confusion matrix of a binary class:

	Predicted Class = No	Predicted Class = Yes
Actual Class = No	True Negative (TN, d)	False Positive (FP, c)
Actual Class = Yes	False Negative (FN, b)	True Positive (TP, a)

Most widely used metric:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Limitations:

- Ignores the cost of misclassification
- Consider an imbalanced 2 class problem:
 - Class 0 has 9,990 examples
 - Class 1 has 10 examples
- If model always predicts class 0, accuracy is 99.9%, but true positive rate is zero. Accuracy is misleading.

We can use a cost matrix. We can define a cost function $C(i|j)$ as the cost of misclassifying class j as class i .

For example:

$C(i j)$	-	+
-	0	1
+	100	-1

Ideally, you need a domain expert to convey this relative cost information, or find a way to determine something reasonable for the trade-off.

Similarly, we can do this in a multiclass example:

$C(i j)$	USA	CAN	AUS	NZ
USA	0	1	10	10
CAN	1	0	10	10
AUS	10	10	0	20
NZ	10	10	20	0

Cost-Sensitive Measures (41:44 - 47:36)

- Precision $p = \frac{TP}{TP+FP}$ (higher = less false positives)
- Recall $r = \frac{TP}{TP+FN}$ (higher = less false negative)
- F-measure $F = \frac{2rp}{r+p} = \frac{2TP}{2TP+FN+FP}$ (higher = less false negatives and false positives)
 - This is a weighted average of precision and recall, so the F-measure will always lie in between them.

```
print('Precision: %0.3f' % precision_score(y_true = y_test, y_pred = y_pred))
```

```
## Precision: 1.000
```

```
print('Recall: %0.3f' % recall_score(y_true = y_test, y_pred = y_pred))
```

```
## Recall: 0.993
```

```
print('F1: %0.3f' % f1_score(y_true = y_test, y_pred = y_pred))
```

```
## F1: 0.997
```

- The above are defined for binary classes.
- To combine them for multiple classes, there are micro and macro.
- Given the metric for the k^{th} class, X_k :

The micro-average weights all instances equally.

$$Precision_{micro} = \frac{TP_1 + \dots + TP_k}{TP_1 + \dots + TP_k + FP_1 + \dots + FP_k}$$

$$Recall_{micro} = \frac{TP_1 + \dots + TP_k}{TP_1 + \dots + TP_k + FN_1 + \dots + FN_k}$$

$$F1_{micro} = \frac{2(TP_1 + \dots + TP_k)}{2(TP_1 + \dots + TP_k) + FP_1 + \dots + FP_k + FN_1 + \dots + FN_k}$$

The macro-average weights all classes equally.

$$X_{macro} = \frac{X_1 + \dots + X_k}{k}$$

Naturally, the difference is much more apparent if there is a class imbalance.

```
pre_scorer = make_scorer(
    score_func = precision_score,
    pos_label = 1,
    greater_is_better = True,
    average = 'micro'
)
```

Reciever Operating Characteristic (47:36 - 60:09)

- Developed in the 1950s in signal detection theory to analyze noisy signals
 - Characterize the trade-off between positive hits and false alarms
- ROC curves plots TP vs. FP
- Performance of each classifier represented as a point on the curve
 - Changing the threshold of algorithm, sample distribution, or cost matrix changes the location of the point
- (0, 0) = declare everything to be negative class
- (1, 1) = declare everything to be positive class
- (1, 0) = ideal (100% true positives, 0% false positives)
- Diagonal line = random guessing for equal number of classes
- Below diagonal = prediction is opposite of true class

If a curve is completely above another, it dominates the other classifier. It may not be this simple, e.g. M_1 is better for small false positive rate, whereas M_2 is better for large false positive rate. We can also take the area under the curve (ideal area = 1, random = $\frac{1}{2}$), but you do always want to look at the curve.

To construct:

- Use a classifier that produces a probability score for each test instance $P(+|A)$.
- Sort the instances according to $P(+|A)$ in decreasing order.
- Apply threshold T at each unique value of $P(+|A)$.

- If $P(+|A) < T$, negative, else positive.
- Count number of TP, FP, TN, FN at each threshold:
 - TP rate $TPR = \frac{TP}{TP+FP}$
 - FP rate $FPR = \frac{FP}{FP+FN}$
- Plot each of these to form the curve

```
mean_tpr = 0.0
mean_fpr = np.linspace(0, 1, 100)
all_tpr = []

cv = StratifiedKFold(n_splits=10).split(X_train, y_train)
for i, (train, test) in enumerate(cv):
    probas = pipe_lr.fit(X_train[train], y_train[train]).\
        predict_proba(X_train[test])
    fpr, tpr, thresholds = roc_curve(
        y_train[test], probas[:, 1], pos_label = 1)
    mean_tpr = np.interp(mean_fpr, fpr, tpr)
    mean_tpr[0] = 0.0
```

Statistical Significance Testing (60:10 - ...)

- Given two models:
 - Model M_1 : accuracy = 85%, tested on 30 instances
 - Model M_2 : accuracy = 75%, tested on 5000 instances
- Can we say M_1 is better than M_2 ?
 - How much confidence can we place on accuracy of M_1 and M_2 ?
 - Can the difference in performance measure be explained as a result of random fluctuations in the test set?
- M_1 is tested on D_1 (size = n_1), found error rate = e_1
- M_2 is tested on D_2 (size = n_2), found error rate = e_2
- Assume D_1, D_2 are independent
- If n_1, n_2 are sufficiently large:
 - $e_1 \sim N(\mu_1, \sigma_1)$
 - $e_2 \sim N(\mu_2, \sigma_2)$
 - $\hat{\sigma}_i^2 = \frac{e_i(1-e_i)}{n_i}$
 - * The estimate of the variance comes from the binomial distribution.
- To test if performance difference $d = e_1 - e_2$ (estimate of mean difference) is statistically significant:
 - $d \sim N(d_t, \sigma_t)$ where d_t is the true difference.
 - Since D_1 and D_2 are independent, their variance adds up:

$$\sigma_t^2 = \sigma_1^2 + \sigma_2^2 \approx \hat{\sigma}_1^2 + \hat{\sigma}_2^2 = \frac{e_1(1-e_1)}{n_1} + \frac{e_2(1-e_2)}{n_2}$$
 - At $(1 - \alpha)$ confidence level: bounds on $d = d \pm Z_{\alpha/2} \hat{\sigma}$
 - * Does this interval include 0?

...

```
# ...

from scipy import stats

invT = stats.t.ppf

print(invT(0.975, df = 10 - 1))

## 2.262157162854099
```